KUBERNETES CKA PREP › MY LEARNING NOTES › SESSION 04

ReplicaSet, Deployment & Service — Why We Never Run Naked Pods

Topic: Deployment / ReplicaSet / Service | CKA Weekend Course · Video 15, 8th Oct | 2026-09-11

The Problem With a Single Pod ("Singleton Pod")

Say you deploy a business app as a single Pod, with resource requests/limits defined, and it's running fine, serving users. Three things can go wrong:

1. **No self-healing.** If that Pod gets deleted for any reason (the node it was on terminated, etc.), nothing recreates it. The application is just down until someone notices and acts.
2. **No scaling.** If user load increases, the single Pod does **not** scale itself. It can only serve up to whatever its `requests` / `limits` allow — past that, it breaks.
3. **No zero-downtime upgrade/rollback.** Bumping the image from v1 → v2 means taking the Pod down — real downtime — with no built-in way back to v1.

> ⚠️ **Conclusion**
>
> In enterprise use, we never run an application directly as a bare Pod.

How We Actually Run Applications: Deployment → ReplicaSet → Pod

```
Deployment → creates a → ReplicaSet → creates → Pod(s)
```

We're still ultimately creating Pods — but through this chain, not directly. Each layer earns its place:

- **ReplicaSet gives:**
  - **Self-healing** — if a Pod it owns is deleted/terminated, the ReplicaSet creates a replacement to get back to the desired replica count.
  - **Scaling** — replica count can go up (manual or automatic) as load increases, and back down when load drops.
- **Deployment gives (on top of ReplicaSet):**
  - **Zero-downtime deployment (ZDD)** — rolling upgrades and rollbacks without taking the app down. This is the main reason we deploy via Deployment rather than a bare ReplicaSet.

Anatomy of a Deployment YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx          # Deployment's own label -- can be anything
spec:
  replicas: 3            # "please maintain 3 copies"
  selector:
    matchLabels:
      app: nginx         # which pods (by label) this RS is responsible for
  template:
    metadata:
      labels:
        app: nginx        # MUST MATCH selector.matchLabels above
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
          resources:
            limits:   { cpu: 100m, memory: 100Mi }
            requests: { cpu: 50m, memory: 50Mi }
```

> ⚠️ **The rule that matters most here**
>
> The **Deployment's own label** (top-level `metadata.labels`) can be anything — it's not load-bearing. But the **Pod template's label** (`spec.template.metadata.labels`) **must match** the **ReplicaSet's selector** (`spec.selector.matchLabels`). If they don't match, the ReplicaSet doesn't know which pods belong to it.

Deploying and Inspecting the Hierarchy

```bash
kubectl create -f deployment.yaml
kubectl get deploy
kubectl get rs
kubectl get pods
```

> 💡 **Naming hierarchy — trace the chain in a busy cluster**
>
> Deployment name gets **prefixed** onto the ReplicaSet name: `nginx-deployment-6fb4f44f88`.
> ReplicaSet name gets **prefixed** onto the Pod name: `nginx-deployment-6fb4f44f88-66phw`.
> From a Pod name alone you can read backward to know exactly which ReplicaSet, and which Deployment, it belongs to.

```bash
kubectl get deploy
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    3/3     3            3           4m24s
webserver           0/4     4            0           8s

kubectl get rs
NAME                           DESIRED   CURRENT   READY   AGE
nginx-deployment-6fb4f44f88    3         3         3       4m27s
webserver-784c968bb9           4         4         3       11s
```

Self-Healing, Demonstrated

Delete one of the ReplicaSet-owned pods — the ReplicaSet immediately creates a replacement to get back to the desired count. **Important:** the new pod comes up with a **brand-new IP**, not the deleted pod's old IP. This is exactly why you can never rely on talking to a pod by its IP address directly in a replica-based setup — which leads straight into why Services exist.

The IP Problem → Why We Need a Service

With 3 replica pods, you have 3 different IPs. The moment one pod is replaced (self-healing, scaling event, node move — anything), its IP changes and whoever was pointed at the old IP breaks. **We never talk to pods directly by IP.**

Enter the Service

- A Service has a **fixed IP** and a **fixed DNS name** — stable for its entire lifetime, independent of whatever pods come and go behind it.
- A Service has a **selector** — the critical rule:

> ⚠️ **Critical rule**
>
> **The selector of a Service must match the label of the Pods** (not the ReplicaSet's selector directly — though in practice, since the ReplicaSet's selector and the Pod template's label are already required to match each other, the Service's selector ends up matching all three).

- A Service has an **Endpoints** object — the live list of `pod-ip:port` pairs currently matching its selector. This list is **automatically kept in sync**: when a pod is deleted and the ReplicaSet self-heals with a new pod (new IP), the Service notices and updates its Endpoints list automatically. Users never have to know or care about individual pod IPs.
- This whole label-driven, auto-updating routing mechanism is what's meant by **SDN — Software Defined Network** in this context.

The Full Label Chain, End to End

```
Deployment.metadata.labels     -> can be ANYTHING, purely cosmetic
ReplicaSet.spec.selector       -> MUST MATCH Pod.template.metadata.labels
Pod.metadata.labels             -> the label actually applied to running pods
Service.spec.selector          -> MUST MATCH Pod.metadata.labels
```

Get any one of these mismatched and the chain breaks silently (a Service with 0 endpoints, or a ReplicaSet that thinks it owns 0 pods and keeps creating duplicates) — this is one of the single most common real-world Kubernetes misconfigurations.

Creating and Verifying the Service

BASH

```
kubectl expose deployment nginx-deployment --name nginx-dep-svc
# service/nginx-dep-svc exposed

kubectl get svc
NAME            TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
kubernetes      ClusterIP   10.96.0.1        <none>        443/TCP   21d
live-svc        ClusterIP   10.111.217.74    <none>        80/TCP    2d4h
nginx-dep-svc   ClusterIP   10.100.200.74    <none>        80/TCP    4s
```

`kubectl expose` is the fast, imperative way to create a `ClusterIP` Service from an existing Deployment — it auto-derives the selector from the Deployment's pod template labels.

BASH

```
kubectl describe svc nginx-dep-svc
# Selector:    app=nginx
# Endpoints:   172.16.10.99:80,172.16.3.166:80,172.16.3.170:80
```

Cross-check those endpoint IPs against the actual running pods:

BASH

```
kubectl get pods -l "app=nginx" -o wide
# NAME                               READY   STATUS    IP              NODE
# nginx-deployment-6fb4f44f88-65kqd   1/1     Running   172.16.10.99    nodeone
# nginx-deployment-6fb4f44f88-vbrln   1/1     Running   172.16.3.170    nodetwo
# nginx-deployment-6fb4f44f88-vdwfv   1/1     Running   172.16.3.166    nodetwo
```

The IPs match exactly — proof the Service's Endpoints list really is just "whichever pods currently match the selector," kept live.

> 💡 **Scope note**
>
> This `ClusterIP` is only reachable **from inside the cluster**. How users *outside* the cluster reach it (NodePort, LoadBalancer, Ingress) is a separate topic for later.

✅ What I Practiced

- [ ] Ran a bare single Pod, deleted it, and confirmed nothing recreated it (`No resources found`) — making the "no self-healing" problem concrete rather than theoretical.
- [ ] Deployed via a Deployment → watched `kubectl get deploy/rs/pods` show the full 3-tier naming hierarchy in one screen.
- [ ] Deleted a ReplicaSet-owned pod and confirmed self-healing kicked in with a **new** pod IP.
- [ ] Created a Service with `kubectl expose deployment ... --name ...`, then cross-verified `describe svc` Endpoints against `get pods -o wide` IPs to prove the selector-matching mechanism live, not just on paper.

Merged into the master reference notes: k8s-master-notes.md — §4 (Workloads) got a "Why Not Run Naked Pods" addendum; §7 (Services & Networking) got a "Service Selector-Matching & the Label Chain" addendum.